

Week 13 Mini-Project: Customer Intelligence Platform

Deploy one ML service and one LLM/RAG service, then run both behind a single production spine. Cloud, MLOps, LLMOps, CI/CD, monitoring, drift, and free-tier engineering.

BUILD	One ML service and one LLM/RAG service that share a single pipeline: versioned data, scripts, tests, CI gate, tracking, monitoring, deployment.
PATH	Azure is the target stack to learn. The recommended default to actually finish is local MLflow + FastAPI + Docker + GitHub Actions + FAISS/Chroma, with one cloud endpoint.
OUTPUT	Public GitHub repo that runs from a fresh clone, live or local endpoints, code-generated monitoring report, and a demo recording.
RULE	Core carries 80, stretch 20. Core-only cannot pass 80. Notebooks are fine for exploration; final execution must run through scripts.

1 · The scenario

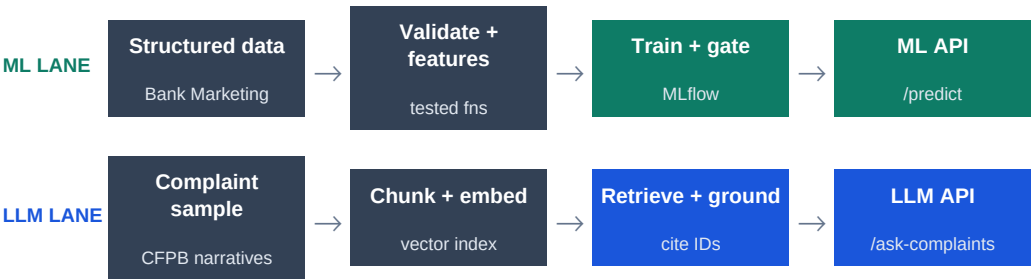
Meridian Financial wants to run smarter outreach campaigns and resolve complaints at scale. It has structured customer data, transaction-like behaviour, and tens of thousands of free-text complaint narratives. Today the ML team builds models in notebooks, the support team reads complaints by hand, and operations has no reliable way to know whether a deployed model has quietly degraded.

Build a small but production-minded version of this platform. The ML service predicts **campaign conversion** (will a contacted customer subscribe). The LLM/RAG service answers **complaint intelligence** questions over real complaint narratives with cited evidence. The graded output is the system around the models: versioned data, reproducible scripts, tested APIs, a CI gate, monitoring, drift checks, and one endpoint that ties both together. If it only runs in a notebook, it is not done.

2 · Why this matters, and what it builds

- **The industry gap this targets.** Companies do not pay for notebooks. They pay for systems that serve, stay monitored, and recover when something shifts. The distance between “trained a model” and “runs reliably in production” is where most ML candidates get rejected, and it is exactly what this project makes you practice.
- **The job-aligned skill.** Wiring a classical ML service and an LLM service into one governed pipeline (CI/CD, a promotion gate, monitoring, drift, cost control, one integration endpoint) is the working definition of an ML, MLOps, or LLMOps engineer. Most candidates can do one modality. Doing both behind one spine is the differentiator.
- **The portfolio signal.** Conversion scoring plus grounded, cited complaint intelligence is the shape of real customer-ops systems. A repo that runs from a fresh clone, with tests, a gate, monitoring, and a drift report, is a stronger interview artifact than any single high-accuracy model that lives only in a notebook.

3 · What you are building



Shared spine + integration: /customer-intel (ML band + complaint themes) · CI/CD gate · model & index registry · monitoring (ML drift + RAG quality) · retrain trigger

4 · Your inputs

Dataset	Use it for	Notes
---------	------------	-------

UCI Bank Marketing	ML model: predict campaign conversion (term-deposit subscription). Default structured choice.	Small and clean. Fast iteration so the system, not feature search, is the focus.
CFPB Consumer Complaint Database	LLM/RAG: grounded Q&A, theme classification, escalation summaries over complaint narratives.	Real public corpus with product, issue, company-response and date metadata. Use a 5,000 to 25,000 record sample locally; full set optional.
Alternatives (optional)	UCI Online Retail for RFM/churn, or Home Credit Default Risk for heavier credit risk.	For a meatier structured problem. Pick one structured set, do not build two.
Synthetic drift layer	Shifted distributions, corrupted records, missing values, or a new complaint theme.	Lets you demonstrate monitoring and retraining without real production traffic.

Scope and data-handling rules (read before you start)

- **Data size:** sample the complaint corpus to 5,000 to 25,000 records. Do not try to ingest 1M rows on a laptop.
- **PII:** complaint narratives are real. Commit samples only, redact obvious personal details, and never show full raw narratives in screenshots or the demo unless cleaned. No keys, no private data, no full datasets in Git.
- **Feature engineering:** keep it simple but production-shaped. Reusable functions, tested transformations, no train-serving skew. Do not burn two days on exotic features.
- **Notebooks:** fine for exploration. Final execution must run through scripts and pipeline commands.

5 · The work

MVP

ship these first; the must-run minimum (~70% of core)

CORE

hardens the system toward 80

Core · MVP (a runnable, defensible minimum)	Tag
Repo + reproducible setup. Public repo with README, the data / src / tests / pipelines / app / monitoring / docs structure, env file and scripts. Define the prediction target and user stories. It must run from a fresh clone.	MVP
Data ingestion. Pull the structured set and a 5k to 25k complaint sample with a download script. Keep only small samples in Git.	MVP
Data validation. Schema, missing-value and duplicate checks plus at least 5 business-rule validations (Great Expectations, Pandera, or your own tests).	MVP
Feature engineering. Reusable, tested feature functions for the structured data; clean complaint text while preserving metadata. Same transform path at train and serve time.	MVP
Baseline + improved model. Train both. Report ROC-AUC, PR-AUC, F1, calibration and confusion matrix, plus a business reading. Track runs and artifacts with MLflow or a metadata JSON.	MVP
ML API. FastAPI with /predict and /health: request/response schema, input validation, model loading, model version in the response.	MVP
RAG complaint assistant. build_index, retrieve, /ask-complaints. Every answer returns retrieved evidence IDs and a one-line evidence-sufficiency note, and refuses when no chunk crosses your similarity threshold. Include a minimum eval of 10 question-answer tests with expected evidence and pass/fail notes.	MVP
CI pipeline. GitHub Actions (or Azure DevOps) runs unit tests and data validation on a sample on every push.	MVP

Minimum modules (keeps the architecture disciplined, not over-built)

ML lane: ingest.py · validate.py · train.py · evaluate.py · serve.py + one CI workflow. RAG lane: build_index.py · retrieve.py · answer.py · rag_eval.py.

Core · hardening	Tag
Dockerize + batch. Containerize the ML service and add a /batch-score endpoint that scores a CSV or JSON batch.	CORE
Promotion gate (relative). The improved model must beat the baseline by a margin you define, or you justify why not. Show one passing run and one deliberately worse model being blocked.	CORE
Integration endpoint. /customer-intel: given a customer plus a product/issue/date filter, return the ML conversion band and the top complaint themes for that segment with cited record IDs. This is what makes it one platform, not two demos.	CORE
Monitoring (both lanes). ML: a code-generated drift report (Evidently or whylogs) on a simulated shift. RAG: retrieval hit-rate, empty-retrieval count, average top-k score, refusal rate, token count and latency.	CORE
Deployment evidence. At least one service cloud-deployed; the other may run locally via Docker Compose if quota blocks you. Evidence: endpoint URL or localhost demo, logs, screenshots, command history.	CORE
Demo + docs. A 5 to 8 minute demo (raw screen recording is fine, no editing expected), one architecture diagram showing data flow, registry/artifact store, API layer, CI/CD, monitoring and retrain trigger, a decision log including rejected approaches, known limitations, and a hardening plan.	CORE

Stretch cluster	Tasks (20 marks total)
MLOps depth	Champion-challenger decision, automated retraining trigger on drift, batch + real-time split with versioned outputs.
Deployment depth	Canary or blue-green with rollback logic, cloud endpoint for the second service.
LLMOps depth	RAG eval harness expanded to 20+ questions with metrics, token and latency guardrails, refusal and adversarial testing.
Governance depth	SHAP with per-prediction explanations saved for 10+ samples, segment error analysis, a PII-redaction audit of the corpus.

Stretch is scored on depth, not count: **up to 5 marks per task** for working, defensible depth, capped at 20. Two strong tasks beat four shallow ones. Attempt stretch only after the core runs end to end.

6 · What “done” looks like

MVP done

Both services answer real requests. POST /predict returns a probability, threshold decision and model version for a valid payload and rejects an invalid one. POST /ask-complaints answers a question with a grounded summary, cited record IDs and an evidence-sufficiency note, and refuses when retrieval is weak. Your 10 RAG tests have recorded pass/fail. CI runs tests and validation on push. The repo runs from a fresh clone.

Full core done

Add the hardening. The ML service is Dockerized with batch scoring. A relative promotion gate blocks a worse model. /customer-intel returns a conversion band plus complaint themes for a filter. A code-generated ML drift report and RAG monitoring metrics exist. At least one service is deployed with evidence. The decision log records what you rejected and why.

Stretch done

At least one cluster is taken to real depth: a champion-challenger decision, a canary with rollback, a RAG eval harness with numbers, cost and latency guardrails, an automated retrain trigger, or governance checks. Depth, with evidence, not breadth.

7 · Reasonable implementation paths

Path	Use when	Suggested stack
------	----------	-----------------

Hybrid (recommended default)	You want enterprise judgement without burning credits or the week.	Local MLflow + FastAPI + Docker + GitHub Actions + FAISS/Chroma + sentence-transformers. Deploy one small API to Azure or GCP.
Azure-first	You have Azure ML, Azure OpenAI and Azure AI Search access.	Azure ML SDK v2, MLflow, Managed Online Endpoints, Azure AI Search, Prompt Flow, GitHub Actions, Azure Monitor.
GCP experiment	You want a deployment comparison.	Cloud Run for FastAPI, Cloud Storage for artifacts, BigQuery or Parquet for batch, GitHub Actions for build and deploy.

Use the LLM in an enterprise-grade way

- Ground every answer in retrieved complaint records. The LLM explains evidence; it does not invent business facts.
- Keep retrieval, prompt construction, generation and evaluation as separate stages so you can debug each one.
- Define RAG confidence with measurable proxies: top-k similarity score and retrieval overlap. Refuse when no chunk crosses the threshold; never invent a confidence number.
- Log prompt version, retrieval parameters, model name, token count, latency and whether generation succeeded.
- Guardrails: no legal or financial advice, no exposure of personal data, no unsupported claims. Use small prompts and caching before blaming the cloud account.

Run the whole thing for zero rupees

- ✓ **Azure for Students** gives roughly Rs 8,000 of credit with no card. Use tiny Azure compute only for a final run; otherwise local CPU or **Colab**.
- ✓ Local **MLflow** or an artifact folder with a metadata JSON still counts, as long as you record model version, dataset hash, metrics and the promotion decision.
- ✓ Swap paid vector search for **FAISS or Chroma** in-process. Persist the index and document mapping or the demo is not reproducible.
- ✓ Keep the repo **public** for unlimited GitHub Actions minutes. Run a smoke train or the eval gate in CI, not full training.
- ✓ Never hard-code API keys. Use environment variables or secrets. Ship a teardown script and run it.

8 · Engineering specs to hit

Promotion gate (relative, not absolute)

Make the logic explicit and tie it to your baseline. Example: promote only if PR-AUC beats the baseline by at least 3 percentage points, F1 drops by no more than 2, and latency stays under your limit. If the improved model does not beat the baseline, do not promote it; record why. For RAG, promote only if retrieval hit-rate and grounded-answer checks improve on your question set.

API contract

Endpoint	Input	Output
GET /health	none	status, model version, vector index version
POST /predict	customer features (JSON)	prediction, probability, threshold decision, model version
POST /batch-score	CSV or JSON batch	scored file path or counts by conversion band
POST /ask-complaints	question + optional filters (product, company, date, issue)	answer, retrieved evidence IDs, evidence-sufficiency note, prompt version
POST /customer-intel	customer features + product/issue/date filter	conversion band + top complaint themes for the segment, with cited IDs
GET /metrics	optional time window	latency, request count, error count, prediction distribution, RAG retrieval stats

9 · How it is scored

Area	Marks	What earns the marks
Framing + repo hygiene	8	Clear target, clean structure, reproducible commands, runs from fresh clone.
Data, validation, features	12	Reusable tested pipeline, schema and business-rule checks, versioned data, no train-serving skew.
ML modelling + tracking	14	Baseline vs improved, correct metrics, MLflow tracking, relative gate decision.
ML serving + deployment	12	FastAPI, validation, health, Docker, batch and real-time, deployment or Docker Compose evidence.
LLM/RAG service	14	Retriever, grounded answer, cited IDs, refusal logic, 10-question eval, theme summarization.
CI/CD + testing	8	Unit tests, data tests, workflow YAML, working evaluation gate.
Monitoring, drift, retraining	8	ML drift report from code, RAG monitoring metrics, retraining or escalation action.
Integration + documentation + demo	4	/customer-intel works, architecture diagram, decision log, demo, limitations, hardening notes.
Stretch	20	Depth in MLOps, deployment, LLMOps, or governance. Scored on quality, capped at 20.

MVP-tagged tasks are roughly 70% of the core marks and are the must-ship minimum. Core alone tops out at 80; stretch is the only route past it. Operational correctness beats a cloud screenshot: a local Docker Compose system with proper CI, validation, versioning and a drift report scores above a broken Azure endpoint.

10 · Minimum acceptance criteria

- The repo runs from a fresh clone using documented commands.
- At least one ML endpoint and one LLM/RAG workflow are demonstrable.
- At least one automated workflow runs through GitHub Actions or Azure DevOps.
- At least one monitoring or drift report is generated by code, not written by hand.
- Every RAG answer shows retrieved evidence IDs and refuses when retrieval is weak.
- The demo shows real inputs and outputs. No secrets, keys, private data, or full datasets are committed.

11 · Submission and sequence

- **By Sunday 23:00** to the LMS: GitHub repo link, architecture diagram, a 5 to 8 minute demo recording, a model report (metrics, confusion matrix, chosen threshold, promotion decision), a RAG report (sample questions, retrieved evidence, failure cases), a monitoring report, and deployment evidence.
- Include **reflection.md** answering the prompts in section 12.

Suggested sequence (do not move ahead until the gate is met)

Stage	Finish first	Gate before moving on
1 Repo + data	Structure, download script, sample data, validation	A teammate can clone and run validation.
2 ML system	Feature functions, baseline + improved model, tracking	Metrics and artifacts logged with a model version.
3 ML API	FastAPI /predict + /health	curl works for valid and invalid payloads.
4 RAG	Index, retrieval, /ask-complaints, 10 eval questions	Answers show evidence IDs and refuse when weak.
5 CI	Tests + validation, then the relative gate	A worse model is clearly blocked.
6 Integrate + monitor	/customer-intel, drift report, RAG metrics	Reports are generated from code.
7 Polish	Docker, deployment evidence, demo, docs	A reviewer can see what works and what does not.

Repository shape

```
customer-intelligence-platform/  
  data/          README with source links, small samples only  
  src/data_pipeline/ ingest.py, validate.py, features.py  
  src/training/   train.py, evaluate.py (relative gate)  
  src/serving/    serve.py (FastAPI), schemas, model loader  
  src/rag/        build_index.py, retrieve.py, answer.py, rag_eval.py  
  tests/         feature, schema, payload, retrieval tests  
  .github/workflows/ CI: tests, validation, eval gate, deploy  
  monitoring/    ml_drift.py, rag_monitor.py, dashboard  
  docs/          architecture, decision log, RAG eval, hardening  
  README.md
```

12 · Reflection (in reflection.md)

- Why this model family and this threshold over the alternatives you weighed?
- What broke first when you tried to deploy, and what did you change?
- Why your gate margin, and what fails if you tighten PR-AUC by another 2 points?
- Show one complaint answer your RAG got wrong or ungrounded. How did your eval or refusal rule catch it, or why did it slip through?
- If this went live to real customers tomorrow, name the one risk you did not fully close.
- What would a senior MLOps engineer criticize first in your repo?

Common blind spots to avoid

- Do not let the LLM answer from the model's prediction alone. It must retrieve and reason over the complaint corpus.
- Do not report only accuracy on imbalanced data. Use precision, recall, F1, ROC-AUC, PR-AUC, threshold analysis and calibration.
- Do not commit full datasets, raw narratives, or laptop-only paths. Use samples, config files and environment variables.
- Do not claim production readiness because an endpoint ran once. That means tests, monitoring, rollback thinking, versioning and failure handling.
- Do not hide failed experiments. A decision log of rejected approaches shows more maturity than a polished but shallow demo.

How this is checked

Your submission gets a short live walkthrough within 48 hours of the deadline. Any function used in scoring may be selected at random for you to explain. You should be able to open any file and say why it is there and what it does. Tools that helped you write code are fine and expected. Code you cannot explain does not count, no matter who or what produced it.